PRIMA-NEXT: patent research foundation and ranked opportunity audit

Repository-specific research plan | 16 October 2026 | Snapshot 4cd51950b22abbab22a0553348df51a36ac20c30

1. Decision brief

The best starting point is a narrow improvement to memory maintenance, retrieval execution or evidence handling. Those areas have identifiable implementation costs and failure modes that can be measured on the existing project. The current code does not establish a new patentable invention. The broad combination of an emotional agent, long-term memory, graph retrieval, planning and reflection is already heavily explored.

Start a short bake-off of R01-R03. Then select at most two experiment tracks. R04 is a useful alternate if the team prefers execution reliability to storage/indexing work. R05-R09 remain meaningful systems research, but either need more implementation or face especially familiar database/runtime solutions. Lower-ranked areas remain on the map so small underlying concepts are not lost.

The ranking is **research priority for this repository**, not a probability of patent grant, a legal opinion that a claim is novel, or an infringement/FTO assessment. Technical eligibility, novelty, inventive step, industrial applicability and disclosure are separate hurdles. A positive technical result may make a valuable paper or capstone contribution while still being obvious or previously disclosed. [Guidelines pp. 9, 16-23, 35-41.]

This report distinguishes four things throughout: implemented source behavior; saved or newly checked evidence; proposed mechanisms; and unverified novelty questions. Proposed gates and examples are plans, not achieved results. No production code was changed for this audit. The highlighted study guide was preserved.

2. Scope, provenance and how to use the files

The structural audit covers **385 owned Python files, 44,720 source lines and 2,735 AST definitions**. It includes all production module roots, owned benchmarks, evaluation, tests, the entry point and the complexity-check script. The per-file CSV gives every file a research family or support disposition, line count, SHA-256, module description, representative symbols and review depth. A support mapping does not mean the file contains an invention.

Every owned Python file was structurally inventoried. Relevant implementations and their canonical callers were read in depth; this is not a claim that every line of every test and export file received a manual semantic review. Non-Python source/configuration, the notebook, documentation, saved results and third-party boundaries are classified separately in the same ledger. Third-party external trees are identified as such and are not counted as team-original programs. Generated caches, model binaries, local secrets and runtime logs are not invention source; their contents were not exhaustively inspected. No secrets were read.

The source authority is the supplied **67-page Indian CRI guidelines PDF**. All "Guidelines p./pp." citations refer to its original PDF page numbers, including its cover as page 1. They do not refer to pages of this report. The annotated plain-language guide is 27 pages with **56 highlights**. H01-H56 identify those annotations in page/annotation order. They express the user's priorities; the original guidelines govern the interpretation. The PDF refers to Annexures I and II but does not include them. This analysis does not

invent their contents. [Guidelines pp. 5, 10, 22.]

Source PDF SHA-256: ac54a08059bf88f0dd144f3e2ac2837b40dc20ea77fffebb197de3280c7bc155. Annotated guide SHA-256: da8e20cebe599d25c4a40428da220001e3b014048fc80f89f9f5bea3145d8f27. Source hashes and page counts were checked on 16 October 2026. Code references use repository-relative paths and one-based lines at the audited snapshot; linked Markdown references open the local source.

Use the main report to choose experiments; the ranking CSV to assign owners and update decisions; and the file ledger to find every module's connection to the research map. The editable Markdown contains the same substantive assessment as this PDF. Each opportunity's "code anchors" points to its current mechanism. Read the full referenced function before implementing a proposal.

3. What actually runs, and what that means for research

The canonical runtime owns trusted contracts, routes, state loading, affect, evidence acquisition, planning, symbolic prediction, uncertainty decisions, bounded correction, optional action, generation/validation and state/memory commit. It also owns the background maintenance lifecycle. There are **six task kinds and five profiles, with ten valid route pairs**, rather than all thirty combinations. Older route-count statements in project documents are historical. [Code: runtime/contracts.py:18; runtime/route_profiles.py:145; runtime/prima_runtime.py:205.]

In the canonical evidence path, retrieval is controlled by the selected profile and reasoning budget. Affect priors exist, but the inspected EvidenceAcquisition request does not pass them into ranking. The separate retrieval controller wrapper and evaluation router must not be confused with the active route. Similarly, multi-turn event segmentation in benchmark processing is different from canonical single-turn memory creation. [Code: workflow/prima_workflow.py:101 and :146; memory/retrieval/retrieval_controller.py:1; evaluation/runners/retrieval_optimization_runner.py:1.]

The system is predominantly deterministic orchestration around existing encoders and model providers. Fixed importance weights, centrality, keyword-based abstraction, symbolic world rules and confidence formulas are not evidence of a new learning architecture. That is useful for reproducible systems experiments: the team can change one mechanism while holding the rest constant. It also narrows what can honestly be claimed. [Code: memory/evolution/semantic_abstraction_engine.py:10; world/predictive_model.py:25; uncertainty/confidence_aggregator.py:1.]

Foreground note admission and background work interact. Adding/updating a note invokes graph indexing; the runtime's maintenance pipeline batches eight admissions and can re-embed, consolidate and evolve memory. The supervisor has a bounded queue and retries, with benchmark barriers for draining work. It is not a durable work broker with proven exactly-once execution across restart. State and memory commits are separate. These call-path facts motivate R01/R06 rather than a generic proposal to "add asynchronous memory". [Code: memory/memory_index.py:52; runtime/prima_runtime.py:376; memory/maintenance/background_supervisor.py:1; workflow/prima_workflow.py:485.]

The implementation already has useful standard optimizations: normalized-vector caches, graph neighbor/degree caches, classifier/overlap caches, shared bounded providers and exact-request campaign response caching. Any new optimization must compare against those existing features. Reintroducing them under a different label would not be a research contribution.

4. Indian CRI tests translated into project decisions

Technical problem and effect. Favor a concrete computer-operation problem: avoid redundant embedding/index work, preserve query consistency during updates, reduce actual model context, or enforce execution/resource bounds. Tie the claimed effect to an implemented mechanism and measurements. A generic processor/memory listing or a useful app result is insufficient. Existing hardware is permissible; novel hardware is not compulsory. [Guidelines pp. 13-15, 20-23, 34-41; H01-H05, H17-H34, H43.]

Separate exclusions. Section 3(k) separately excludes mathematical methods, business methods, algorithms and computer programs per se. Do not treat a scoring formula, a workflow sequence or an AI label as the invention. Examine the claim as a whole, the actual implementation and its technical contribution. Section 3(m) mental/game rules and 3(n) presentation of information also matter. Business-method substance is not rescued by software implementation. [Guidelines pp. 9-15, 23-43; H02-H03, H35-H41.]

Novelty. Define the proposed combination and compare every essential feature against the relevant disclosure of one earlier reference at a time. Record exact passages, dates and reasons. Do not establish lack of novelty merely by combining unrelated documents; combining references belongs to the separate inventive-step analysis. Do not infer novelty because a keyword search found no identical title. The guidelines give a seven-step novelty approach. [Guidelines pp. 16-17; H12-H13.]

Inventive step. Identify the skilled person, common knowledge, inventive concept, differences from prior art and whether those differences would be obvious without hindsight. Technical advance or economic significance does not replace the non-obviousness requirement. A familiar cache plus a familiar queue plus a familiar graph is not automatically inventive. Test whether a plain standard solution explains the entire improvement. [Guidelines pp. 17-19; H14-H15.]

Industrial applicability. Explain a workable technical deployment and how the result can be made or used. For this project a reproducible memory/retrieval subsystem for deployed software is a more concrete starting point than claims about generalized cognition. This remains a separate requirement from novelty. [Guidelines p. 20; H16.]

Sufficiency and best method. The description must teach how the components interact, the concrete steps and the best implementation known. Explain invalidation, thresholds, state ownership, failure behavior and why the measured effect follows. Merely listing an encoder, database, agent and feedback module reproduces the disclosure weakness illustrated by Motupalli. [Guidelines pp. 20-23, 50-53; H04-H05, H43.]

AI disclosure depends on the contribution. A learning invention needs the relevant architecture, training and parameters; a data/preprocessing invention needs the essential dataset traits, transformations and reproducible process. Architecture, training, preprocessing and downstream hardware/system interactions must be specified when essential to the claimed effect. RL needs the relevant state/action/reward design. The document generally permits describing defining dataset characteristics rather than handing over an enormous or sensitive dataset, but essential reproducibility details cannot be hidden. [Guidelines pp. 53-61; H07-H11, H44-H50.]

Human inventorship. Keep a human contribution record for the proposed technical mechanism. The guidelines distinguish AI-assisted human inventions from autonomous or very limited human-intervention AI-generated inventions. Contributors, inventors, applicants and owners are not automatically the same people. An AI-generated idea list is not proof of human inventorship. [Guidelines pp. 49-50; H06, H42.]

Modern labels. Prompt engineering, RAG, agents, memory and fine-tuning must still pass the same legal and disclosure tests. H51-H56 are reading aids in the guide, derived from the original's technical-effect and AI-disclosure treatment; they are not additional statutory rules or categorical approvals. A new name for the stack does not supply a technical mechanism. [Guidelines pp. 34-43, 49-61.]

The quantum and blockchain examples on pp. 61-66 do not describe implemented PRIMA features. They illustrate disclosure of actual technical integration; bolting these labels onto the project would not strengthen its research position. Page 67's administrative applicability does not change the above patentability requirements. The Act and Rules prevail over guidelines. [Guidelines pp. 5, 61-67.]

5. Ranking method

Ordering considers: identifiable technical effect; specificity of a testable mechanism; access to code and experiments; effort to obtain discriminating evidence; and closeness of earlier work. It is an ordinal judgment, not a numerical patentability score. All opportunities have **unproven novelty and non-obviousness**. "High technical" means a plausible connection to computer operation, not legal eligibility already established. "Readiness" means how much of an experimental seed exists, not how mature the invention is.

Tier A is the first bake-off. Tier B is a credible alternative or later systems track. Tier C is conditional or heavily crowded and needs a sharper distinction. Tier D is presently support work or a weak standalone patent direction. Within a tier, prefer a smaller well-tested mechanism over a larger unverified architecture claim. A lower-ranked idea can move up when a specific unexpected effect and a defensible prior-art distinction are demonstrated.

All gates below are proposed initial criteria. Pre-register final tolerances before test-set access, adapting them to pilot variance, the practical deployment and available hardware. Do not adjust them after seeing results merely to make a candidate pass.

6. Ranked shortlist and full research map

A/B/C/D indicate research priority only. Every row has unproven novelty. The CSV has editable ownership and decision fields.

Rank	Research family	Tier	Technical fit	Readiness	Crowding
R01	Dependency-aware incremental memory maintenance	A	High	Medium	High
R02	Budgeted retrieval scheduling with safe candidate reuse	A	High	Medium	High
R03	Source-span and condition-preserving context compression	A	High	Medium	Very high
R04	Outcome-grounded bounded correction control	A	Medium-high	Medium	Very high
R05	Version-safe online embedding and representation migration	B	High	Low-medium	High
R06	Recoverable commits across state, memory and derived indexes	B	High	Low-medium	Very high
R07	Measured resource scheduling with output-equivalence checks	B	High	Medium	High
R08	Utility- and cost-aware memory admission	B	Medium-high	Medium	Very high
R09	Dependency-preserving retention and forgetting	B	Medium-high	Medium	Very high
R10	Conservative identity resolution with session provenance	B	Medium	Medium	High

Rank	Research family	Tier	Technical fit	Readiness	Crowding
R11	Event-time temporal retrieval and validity tracking	C	Medium-high	Low-medium	High
R12	Calibrated confidence for compute and abstention decisions	C	Medium	Medium	Very high
R13	Postcondition-verified procedural memory and reuse	C	Medium-high	Low-medium	Very high
R14	Enforceable tool-effect and cancellation budgets	C	High	Low	High
R15	Affect uncertainty as a bounded compute-control signal	C	Conditional	Low	High
R16	Adaptive event segmentation with raw-source fidelity	C	Medium	Medium	High
R17	Query-conditioned graph paths with explicit support	C	Medium-high	Medium	Very high
R18	Support-checked semantic abstraction and lineage	D	Conditional	Low-medium	Very high
R19	Emotion encoder and calibration pipeline	D	Conditional	Medium	Very high
R20	Planning and symbolic world constraints	D	Conditional	Medium	Very high
R21	Typed routes and trusted/untrusted prompt boundaries	D	Medium	Medium	Very high
R22	Evaluation integrity and replay accounting	D	Conditional	Medium	High
R23	Provider plumbing, configuration, events and monitoring	D	Low standalone	High for reuse	Very high
R24	Third-party assets, fixtures, exports and project documents	D	No team-original mechanism established	Support only	Not assessed as team IP

7. Module coverage and implementation implications

This matrix accounts for every owned Python root. The ledger expands it to every file; interdependent research families overlap, so their counts must not be added as separate inventions.

Module	Files / lines	Research links	Assessment
runtime	8 / 2,096	R21; R01; R06	Canonical lifecycle, dependency injection, ten valid routes and maintenance ownership. Strong as the integration surface for a narrow mechanism; generic orchestration alone is weak.
workflow	11 / 2,188	R04; R03; R06; R13	Evidence acquisition, correction reinsertion, generation/validation and commits. Trace this caller before attributing an effect to a dormant helper.

Module	Files / lines	Research links	Assessment
memory	65 / 5,269	R01-R03; R05; R08-R11; R16-R18	Largest research surface: repository/index, retrieval strategies, representation, event/identity processing, maintenance and evolution. The per-file ledger resolves every submodule.
affect	23 / 1,673	R15; R19	Existing encoder/legacy classifier, PAD/momentum/history, salience and hints. Technical control effect is not demonstrated by the canonical retrieval path.
reasoning	12 / 842	R02-R04; R17	Bounded deterministic evidence acquisition, integration and sufficiency. Lexical coverage is not entailment; isolate cost and unsupported-answer behavior.
planning	8 / 1,241	R20; R04	Deterministic goals, action selection, plan evaluation and replanning. Supports a verified execution experiment rather than a broad cognition claim.
reflection	17 / 1,409	R04; R13	Triggers, acceptance, adapters, templates, lineage and rule extraction. Separate actual outcome improvement from heuristic confidence.
action	6 / 578	R13; R14	Sequential execution, policy and audit records. Success status is distinct from a verified external postcondition.
tools	6 / 544	R14; R13	Routing, registry, schema validation and execution. Real isolation/cancellation/effect semantics must be established before stronger safety claims.
world	7 / 799	R20	Symbolic predictions and simulation context. No learned physical-world predictor established.
uncertainty	7 / 949	R12; R04	Static aggregation/calibration profiles and execution gate. Fit/validate real outcome probabilities before using confidence as evidence.
state	4 / 328	R06; R10; R15	Cognitive/emotional state and optimistic process-local ownership. Scope, concurrency and persistence boundaries matter.
events	8 / 463	R23; R06	In-process events, publisher/subscriber and maintenance event contracts. Useful instrumentation, no durable broker guarantee.
security	7 / 180	R21; R14	Allowlist, sanitation, redaction and validation support. Regex and labels are not formal isolation.
llm	8 / 766	R23; R21; R03	Provider transport, schema/generation contracts, parsing, prompt assembly and rate limits. Reuse for controlled experiments.
config	5 / 153	R23	Runtime mode/environment/settings. Parameters define the baseline; parameter changes alone are weak research.
monitoring	5 / 137	R23; R22	Health/metrics/tracing. Measure actual work and overhead; instrumentation is enabling infrastructure.
evaluation	29 / 5,856	R22; R02; R04; R12	Metrics, synthetic generators and ablation runners. Distinguish analytic proxies from real measurements; preserve gold outside runtime.

Module	Files / lines	Research links	Assessment
benchmarks	59 / 10,152	R07; R19; R22	Owned adapters and campaign, result/provenance tooling and emotion training. Separate third-party data and supplied/oracle contexts.
tests	88 / 8,974	R22; all families	Regression evidence and fixtures. Structural coverage is complete; a test file is not proof that its tests were run in this audit.
scripts	1 / 99	R24	Complexity-check entry point. Support only.
main.py	1 / 24	R23	Entry-point wiring. No independent technical mechanism established.

Non-Python scope: notebook cells and campaign/configuration source, repository setup/CI/license metadata, project explanations/presentations, saved results and benchmark fixture/data boundaries. Documentation can suggest a question but cannot override current source. Third-party models/datasets and external submodules require provenance, not team invention credit. Logs/caches/local secrets/generated outputs are excluded from exhaustive invention review.

8. Evidence already available, and what it does not establish

Saved embedding ablation artifacts describe one conversation and 197 queries, seed 13, top-k 30, subset_validation enabled. Reported recall@5 / MRR were: stable 0.010152 / 0.010081; MiniLM 0.157783 / 0.110655; BGE 0.157360 / 0.138600; Nomic 0.329103 / 0.254356. These are saved subset results, not newly executed full-benchmark measurements. Their recorded hardware/software differs from the current project environment. [Code/artifact: evaluation/results/final/embedding_ablation_results.json.]

The saved representation ablation on the same stated 197-query setup reports raw Nomic recall@5 0.403130 and MRR 0.323311; semantic 0.329103 / 0.254356; event 0.355753 / 0.268545. The raw representation performs better on these reported retrieval metrics. This is valuable negative evidence against assuming structured representations help automatically. It does not establish which mechanism caused the difference or its generality. Recorded execution times are not a controlled current hardware latency study; storage_overhead values of one are not measured disk usage. [Artifact: evaluation/results/final/representation_ablation_results.json.]

Several historical outputs and release-readiness transcripts exist, but a saved pass statement is not a fresh test run. A focused five-file pytest attempt in this audit returned no completed result and was interrupted. No fresh integration-suite pass is claimed. The actual compressor class/function body was separately executed in isolation with a minimal evidence fixture: four boundary/behavior checks passed, including the condition-loss example. This does not validate the rest of the runtime or estimate answer quality.

Some existing metrics are proxies: retrieval_scaling in phase5_scientific_metrics computes a latency formula; the representation serialization_consistency check compares a serialization to itself; correction utility rewards changes/confidence; word-removal compression_loss is not semantic loss. Preserve these as diagnostics if useful, but do not present them as measured hardware performance, reproducibility, causal utility or faithfulness. [Code: evaluation/metrics/phase5_scientific_metrics.py:340; memory/semantic_representation.py:1; workflow/correction_loop.py:115; memory/retrieval/context_compressor.py:1.]

9. Common experiment protocol

Freeze the unit of comparison. Record commit, code/config hashes, dataset revision/license, case IDs and split manifest, provider/model/tokenizer revisions, prompt/schema, seed, generation settings, context/retrieval/correction budgets, backend, hardware and cache regime. Give each run a unique identity and retain raw per-case traces. Compare the same questions, not just the intersection of whatever happened to finish.

Split before tuning. Use conversation/source-disjoint development and locked test sets for memory research. Keep all questions from one conversation in the same split. Use official train/dev/test separation for GoEmotions. Tune thresholds, scheduler parameters, model selection and acceptance rules on development data only. Do not feed gold answers/support labels into the runtime. Use synthetic cases for controlled faults, then real held-out data for generalization.

Use a baseline ladder. B0 is the existing code/configuration; B1 fixes correctness/instrumentation needed for a fair test; B2 is the simplest standard mechanism, such as memoization, sentence extraction, time-based decay, a transactional outbox or fixed retry; B3 is a compatible published method; N is the proposed change. Keep a result against B0, but assess research value chiefly against B2/B3. Charge extra encoders, rerankers, teacher calls, offline processing and migration storage to the proposed system.

Separate subsystem and end-to-end results. First measure actual primitive work: similarity comparisons, embeddings, graph expansions/writes, model calls, token counts, I/O and elapsed time. Then measure final QA, source support and reliability on the canonical route. A faster index that degrades answers, a better score that adds expensive calls, or a benchmark-only path that is not used in runtime does not establish the intended result.

Use the real tokenizer and clock. Record monotonic wall time and p50/p95, with p99 only when the sample size supports it; report throughput separately from latency. Include warm-up policy, cold/warm runs, cache hits and model loading. Measure RSS and actual stored bytes; record VRAM where relevant. Report energy only if actually measured with a documented sensor/method. A formula or a lower token estimate is not a measurement of hardware improvement.

Count every outcome. Retain timeouts, parse failures, missing evidence, abstentions, retries and failed cases in the declared denominator. Also report quality conditional on valid responses, clearly labeled. Existing paired-comparison tooling can exclude unusable pairs, so publish the excluded-case counts and reasons rather than hiding them behind a shared-case result. Distinguish model-only, simple-RAG, full PRIMA and each ablation.

Use appropriate benchmark meanings. LoCoMo/LongMemEval support long-memory and update questions. HotpotQA modes must identify supplied distractor context, a named retriever or oracle support; supplied context is not open-domain retrieval, and oracle support is not a headline system result. Emotion component scores are different from end-to-end runtime value. Fake providers and tiny fixtures prove wiring/recovery, not accuracy or speed on a real model.

Quantify uncertainty honestly. Run at least three independent seeds or repeated schedules for a pilot where randomness exists; use more if variance demands and budget permits. Cluster-bootstrap paired differences by conversation/source for long-memory results, rather than treating correlated questions as independent. Report sample counts and intervals, not just percentage gains. Fixed deterministic fixtures test invariants; they do not need pretend statistical intervals.

Use adversarial correctness fixtures. Include negation/conditions, source contradictions, out-of-order edits, duplicate ingestion, retracted support, multi-person pronouns, session switches, concurrent writes, restart at commit boundaries and synchronous blocking tools where applicable. Keep these small and understandable. The proposed mechanism must state which invariants it guarantees and which are evaluated empirically.

Ablate the claimed mechanism. If R01 claims dependency scheduling plus selective consistency, remove each separately. If R02 claims cost control plus reuse, compare reuse alone. If R03 claims

support closure, compare whole-sentence extraction. If R04 claims verified correction, compare fixed retry and score-only acceptance. A combined stack improvement cannot identify the inventive contribution by itself.

Stop or revise. Kill a patent track when its effect disappears with fair accounting, a plain standard baseline matches it, the claimed feature combination is already disclosed, or enabling it requires unsupported hidden assumptions. Preserve negative findings for the capstone. A paper-quality result and a patent-quality distinction are different outputs.

10. Specialized execution plan for the team

Use four roles, combining roles if the team is smaller: systems owner (memory/index/runtime), evidence owner (retrieval/compression/correction), evaluation owner (datasets, traces, statistics), and research/IP owner (full-paper/claim charts, disclosure chronology). Everyone records their own technical contribution. The evaluator should review metrics and splits independently of the person proposing a mechanism.

Days 1-3: lock the foundation. Confirm the audited baseline and environment, resolve any test execution issue, select the hardware, pin datasets/models, establish one run manifest and instrument actual work. Trace canonical routes and document source/event/session/version ownership. Produce a discrepancy list for stale docs and proxy metrics. No new architecture is required at this stage.

Days 4-7: reproduce and expose. Run small real baselines, reproduce the saved representation comparison if resources allow, and build R01 update-consistency, R02 duplicate-call and R03 condition/attribution fixtures. Add a simple standard baseline for each. In parallel, read the full closest references and patent families; fill feature-level charts. Deliver three two-page candidate briefs, baseline traces and an explicit unknowns list.

Week 2: bake off R01-R03. Spend a bounded, comparable pilot budget per candidate. Implement only the smallest differentiating mechanism in an experimental branch. Test actual costs and correctness, not an entire new agent. R01 needs edit/deletion/update traces; R02 needs cost/reuse/invalidation traces; R03 needs original-source support labels. Pick at most two tracks using the registered gates and prior-art review. R04 can replace a failed candidate if its outcome labels are available.

Weeks 3-4: establish mechanism and effect. Complete the selected mechanism, failure handling and ablations. Scale note counts/workload pressure gradually. Compare B2/B3 and report negative cases. Keep the generator, embeddings and budgets controlled; do not tune five other parts of the stack to rescue a candidate. Produce a working technical description with diagrams, pseudocode and parameter choices that another team member can reproduce.

Weeks 5-6: holdout and stress. Lock the candidate configuration, then run the held-out test, conversation-clustered analysis and restart/update stress tests. Repeat on a second workload or model only to assess the claimed scope. Archive raw per-case outputs, measurements and exact environment. A second person should replay a representative subset using only the written recipe.

Weeks 7-8, if justified: disclosure decision. Refresh the prior-art search using the actual final mechanism, not the original brainstorm title. Prepare feature charts, technical-effect tables, human contribution records and the best-method description. Take that concrete packet to the college's IP contact/patent professional before deciding whether and what to file or publicly disclose. If no defensible distinction remains, deliver the reproducible capstone result without forcing a patent narrative.

This is a sequence, not a completion guarantee. Dataset access, hardware, baseline reproduction and prior-art findings can change its duration. No full GPU campaign was run during this audit, and no projected deadline is presented as a measured completion time.

11. Candidate-specific work packages and interfaces

R01 work package. The systems owner maps source-note versions to embeddings, edges and summaries; the evaluator defines update/retraction truth. The first prototype should use the current repository and current batch-eight runtime. Produce a dirty-set trace and query visibility rule before introducing a new database. Acceptance includes deletion/invalidation behavior and total maintenance work, not only enqueue speed. R06 recovery can follow after the performance mechanism is discriminated.

R02 work package. The evidence owner first implements ordinary request-local seed reuse in an experimental controller. The evaluator measures whether it already explains most savings. Only then test a marginal-cost scheduler. Query revision, corpus updates and representation versions form the reuse boundary. Avoid changing fusion/reranking simultaneously; otherwise the cost/quality attribution becomes unclear.

R03 work package. Build a small adjudicated set of original spans with conditions, negations, temporal qualifiers and dependencies. The evidence owner starts with extraction, actual tokens and offsets. The evaluator scores support against the original, not the compressed text. Report abstentions and the limits of automated entailment scoring; use human checking where model-based metrics disagree.

R04 alternate. The evidence/evaluation owners collect before/after outcomes and separate a supported correction from a cosmetic rewrite. Begin with retrieval corrections; add tool actions only after R14-style execution boundaries and effect identities are credible. Success must survive equal-call comparisons.

Avoid making R01, R02 and R03 prerequisites for one another. Each must have a standalone baseline and effect. Their combined stack can be a later integration experiment; it is not a shortcut to proving each component novel.

12. Prior-art work still required before a patent decision

The screen below uses primary research abstracts/metadata and selected patent publication claims. It establishes crowding and comparison targets, not an exhaustive novelty search. Research articles can be prior art regardless of whether their authors patented anything. Patent publications from another country can be relevant disclosures; the existence of a publication does not itself prove an enforceable Indian right or infringement.

For each surviving mechanism, inspect full papers, implementation repositories, patent claims, detailed descriptions, drawings, family members, citations and earliest public disclosures. Verify dates from the actual version/publication. A priority date is not automatically the public-disclosure date. Preserve a copy or hash and exact passage references. Search by technical features and synonyms, not just PRIMA or RAG.

R01 search clusters: incremental/materialized views; dependency invalidation; graph consistency; query snapshots/frontiers; embedding reuse; budgeted maintenance; deduplicated derived memory. R02: adaptive/multi-stage retrieval; budgeted search; shared candidate workspaces; cache invalidation; marginal utility; query planning. R03: evidence/context compression; source attribution; provenance spans; dependency closure; negation-preserving extraction; token-budget selection. R04: verified retries; correction acceptance; outcome-based control; reflection; side-effect budgets; rollback. Search relevant patent classifications and cited/citing families after an initial feature search.

Use this exact chart structure: **essential feature; its technical role; implemented/proposed code; reference ID and public date; exact claim/paragraph/page; present/partial/not yet established; difference; why the difference is not a routine choice; evidence needed**. "Not yet established" must not silently become "absent". For novelty, assess whether one earlier disclosure teaches all essential features of the proposed combination. For inventive step, separately consider common knowledge, motivation and obvious combinations.

A provisional working mechanism statement can be written as: "When [specific technical trigger], [implemented components] perform [specific operations using defined state/version data] to achieve [measured technical effect], while enforcing [correctness/resource constraint]." This is a research articulation aid, not a drafted claim or a conclusion that the mechanism is patentable. Avoid result-only language such as "an intelligent system that optimizes memory."

13. Disclosure, ownership and contribution record

Record when the relevant mechanism was conceived by humans, implemented, tested and first disclosed. Check public repository visibility/history, presentations, reports, demonstrations, theses, videos, preprints and conference submissions. The local repository has a GitHub remote, but public availability and dates were not independently established in this audit. Local commit dates alone do not prove a public disclosure or a filing priority date.

The repository uses Apache-2.0. Its contributor patent grant is limited by the license terms to necessarily infringed claims of contributions or their relevant combination; it is not evidence that the team owns every patent or that all possible future inventions were licensed. Review the actual contribution/disclosure situation with the college IP policy and the relevant owners. Do not assume a grace period or that deleting a public file erases disclosure. [Repository LICENSE and CONTRIBUTORS.md; Guidelines pp. 16-19, 49-50.]

Keep an inventor/contribution log with dated human explanations of the technical problem, proposed mechanism, rejected alternatives and experiments. Separate implementation effort from the inventive contribution assessment. Record AI assistance and which human supplied/modified the technical conception; do not list an AI as an inventor based on this report. No ownership determination is made here.

For the final packet, include a plain-language problem statement; the closest standard baseline; system/data-flow and failure diagrams; exact pseudocode and best implementation; version/data structures; dataset-defining traits and splits; training recipe if learning is essential; measured technical effects and limits; ablations; full prior-art charts; and a reproduction recipe. Negative results and known failure cases make the disclosure more credible and help avoid an overbroad claim.

14. Opportunity charters

Each charter identifies a small proposed mechanism, a standard alternative, a falsifiable technical effect and original-guideline pages. R01-R24 are research IDs, not patent numbers. A source-code anchor identifies implementation evidence; proposed features are explicitly labeled.

R01. Dependency-aware incremental memory maintenance

Tier A | Technical fit: High | Readiness: Medium | Prior-art crowding: High | Effort: Medium

Code anchors: memory/graph/graph_builder.py:17; memory/graph/graph_builder.py:48; memory/maintenance/maintenance_pipeline.py:1; memory/evolution/memory_evolution_engine.py:1; runtime/prima_runtime.py:376.

Implemented now. Graph rebuild compares all pairs within a memory type; indexing one note scans the repository. Maintenance re-embeds admitted notes and updates the index before consolidation and evolution. The production runtime batches eight admissions; it does not use the pipeline default of one. Evolution can create another semantic note from an eligible community without an evident cluster-fingerprint deduplication rule. Existing graph neighbor and degree caches already reduce some read work.

Problem to investigate. Maintenance work can grow faster than useful changes. A lower write count alone is insufficient if the query reads stale edges, outdated summaries or incompatible embeddings. The present graph API adds/updates edges but exposes no corresponding deletion operation, so changed or forgotten content needs a consistency audit.

Proposed mechanism. Maintain a versioned dependency set from source note to embedding, edges and derived summaries. On an edit, compute the minimal affected closure, invalidate dependent artifacts and process it within a work budget. A query receives a snapshot frontier and either waits only for dependencies it needs or explicitly excludes invalid artifacts. Reuse unchanged embeddings and deduplicate derived summaries by source versions. This combination is a proposed experiment, not current functionality.

Plain example. Changing one fact in a 10,000-note memory should update the relevant neighbors and summaries. A question about that fact must not receive its old summary while unrelated questions proceed.

Experiment and baselines. Replay identical append, edit, retract and burst workloads at 1k, 10k and 50k notes, extending scale only if hardware permits. Compare current batch-eight maintenance, a straightforward incremental index with fixed batches, periodic full rebuild and the proposed dependency controller. Compare a compatible graph-memory baseline such as LightRAG separately; disclose differences in extraction and model use.

Metrics and ablations. Actual similarity comparisons, embedding calls, edge/summary writes, CPU time, peak RSS, queue age, p50/p95 admission and query latency, obsolete-answer rate and retrieval/answer quality. Sweep locality, update rate and queue pressure; ablate dependency tracking, selective waiting and summary deduplication independently.

Proposed go/no-go gate. Initial target: at least 30% fewer measured comparisons or embedding calls and 20% lower p95 query latency during maintenance, with no stale answer on the adversarial consistency fixtures and no more than one percentage point loss in the declared quality metric. These are proposed decision thresholds, not achieved results. If ordinary incremental indexing matches the result, keep the engineering improvement and abandon the broader invention story.

First task. Instrument one maintenance trace and build edit/retraction fixtures before designing a controller.

Disclosure needed. Specify dependency keys, version transitions, invalidation, deletion semantics, snapshot visibility, budget accounting, recovery and the best implementation. Show where the mechanism changes storage/processor operation.

Guideline connection: original pp. 16-23, 35-41, 58-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P02, P09, P14, P15, X01, X02. Novelty and non-obviousness remain unproven.

R02. Budgeted retrieval scheduling with safe candidate reuse

Tier A | Technical fit: High | Readiness: Medium | Prior-art crowding: High | Effort: Medium

Code anchors: memory/retrieval/retrieval_controller.py:1; memory/retrieval/graph_strategy.py:1; memory/retrieval/retrieval_router.py:1; evaluation/runners/retrieval_optimization_runner.py:1; workflow/prima_workflow.py:146.

Implemented now. The full profile runs dense, sparse, temporal and graph retrieval in sequence. Candidate pool size is at least 30 and six times top-k. Graph retrieval obtains its own dense/sparse seeds, potentially repeating work. Fusion uses fixed weights. The keyword-based adaptive router is referenced by evaluation and tests, rather than driving the canonical controller. Sparse retrieval scans notes; it is not an inverted BM25 index.

Problem to investigate. Every question pays for a fixed bundle even when an inexpensive strategy is sufficient. Simple seed reuse or an inverted index is an important comparator but is routine optimization by itself.

Proposed mechanism. Use a request-scoped evidence workspace with query, corpus, representation and filter version keys. Estimate each next strategy's marginal evidence gain against its measured cost; reuse valid seeds and stop when a declared evidence sufficiency constraint is met. Invalidate reuse after query revisions, state/filter changes or corpus updates. Preserve a full-retrieval fallback for uncertain cases.

Plain example. A direct date lookup might use sparse retrieval once; a two-hop relationship question could spend its remaining budget on graph expansion using already computed seeds.

Experiment and baselines. First measure duplicate calls in the canonical route. Compare fixed full retrieval, dense+sparse only, ordinary seed memoization, the existing router wired into an experimental route, and a cost-aware scheduler. Use held-out LoCoMo conversations plus HotpotQA in explicitly named context modes. Match candidate/context budgets and representation backends.

Metrics and ablations. Retrieval recall@k/MRR, supporting-fact coverage, final QA, p50/p95 latency, candidates scored, encoder calls, graph expansions and actual CPU/GPU time. Ablate reuse and scheduling separately; test a corpus update between hops and adversarially expensive queries.

Proposed go/no-go gate. Proposed gate: 20% fewer measured retrieval operations or p95 latency with quality within one percentage point of fixed full retrieval. The gain must persist beyond memoization and a simple keyword router. Otherwise this is useful performance work with a weak patent distinction.

First task. Add call-count tracing around the existing controller without changing ranking, and benchmark repeated seed work.

Disclosure needed. Describe state representation, cost estimation, scheduling decisions, invalidation keys, stopping/fallback conditions and actual provider/token budgets. An isolated scoring formula is not the proposed invention.

Guideline connection: original pp. 16-23, 35-41, 58; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P05, P06, P07, P08, P10, P19. Novelty and non-obviousness remain unproven.

R03. Source-span and condition-preserving context compression

Tier A | Technical fit: High | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: memory/retrieval/context_compressor.py:1; reasoning/evidence_integrator.py:1; workflow/prima_workflow.py:146; llm/prompt_builder.py:1; workflow/output_validation.py:45.

Implemented now. The compressor keeps ranked evidence and truncates its final boundary by whitespace words. It retains evidence IDs and a citation map, including entries for dropped evidence. Its compression_loss measures removed words, not semantic loss. The output validator checks structural source references; it does not prove that a cited source entails a generated answer.

Problem to investigate. An intact citation ID can conceal an incomplete proposition. An isolated execution of the actual compressor body changed "Valve is open only when sensor is not faulty" to "Valve is open" at a three-word budget. This demonstrates a mechanism limitation, not its population frequency or a measured failure rate.

Proposed mechanism. Represent candidate evidence as source spans with dependencies for conditions, negation, entity identity and validity time. Select the smallest sufficient support closure using the generator's real tokenizer; attach original offsets and versions. If a closure cannot fit, retrieve a shorter valid support or abstain. Test extractive closure before proposing generative rewriting or a second model.

Plain example. Retain "only when sensor is not faulty" together with the valve statement, or refuse to treat the truncated sentence as evidence for unconditional opening.

Experiment and baselines. Compare current word-prefix truncation, tokenizer-prefix truncation, sentence extraction, a compatible LongLLMLingua implementation and the proposed closure selector. Test several actual token budgets with a frozen generator. Use negation, conditions, exceptions, pronouns and conflicting temporal updates, plus normal benchmark questions. Check answers against original source spans, with blinded human review of a stratified subset.

Metrics and ablations. Actual input tokens and generation time, answer quality, original-source support precision/recall, unsupported-claim rate, attribution correctness, abstention and support-closure completeness. Report both quality and token cost; a citation ID survival rate is insufficient.

Proposed go/no-go gate. Proposed gate: reduce actual context tokens by at least 25% versus uncompressed retrieval while staying within one point of QA and not worsening original-source support; additionally beat sentence extraction on either cost at matched support or support at matched cost. If the effect follows an existing compressor or provenance scheme, do not describe it as new.

First task. Turn the isolated condition-loss probe into source-span fixtures and add real tokenizer accounting in an experimental harness.

Disclosure needed. Disclose span extraction and dependency rules, source offsets/hashes, selection procedure, overflow behavior, tokenization, entailment evaluation limits and failure examples.

Guideline connection: original pp. 20-23, 35-41, 53-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P11, P20. Novelty and non-obviousness remain unproven.

R04. Outcome-grounded bounded correction control

Tier A | Technical fit: Medium-high | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: workflow/correction_loop.py:115; workflow/orchestration_engine.py:229; reflection/reflection_engine.py:1; reflection/reflection_acceptance.py:1; evaluation/metrics/reflection_harm.py:1.

Implemented now. Correction has hard budgets, accepted advice types, duplicate signatures and leakage checks. Its utility uses positive confidence change or a 0.25 bonus if query, plan or answer changed. That rewards activity without demonstrating improvement. Several reflection signals are deterministic text/threshold heuristics. Evaluation already contains useful before/after harm accounting.

Problem to investigate. A changed answer and higher internal confidence can still be wrong. Repeating a transient tool action can also repeat a side effect. Generic reflection, retry-until-score and experience memory have extensive earlier disclosure.

Proposed mechanism. Select a correction action using observable failure evidence and an action-specific resource/effect budget. Require an independently checkable acceptance condition, such as restored required support or a verified tool postcondition; keep the previous answer/evidence available for rollback. Learn or calibrate action value only on development data. Stop on no new evidence, budget exhaustion or an unsafe effect state.

Plain example. A second retrieval is accepted because it supplies the missing source for a date claim, not because the same confidence formula rose after rewriting the answer.

Experiment and baselines. Use paired same-question runs with no correction, current proxy utility, fixed one-retry, CRAG-style retrieval correction and the proposed controller. Measure retrieval-only, regeneration and tool corrections separately. Freeze generation settings and provide the same starting evidence. Evaluate correct-to-wrong and wrong-to-correct transitions; oracle labels stay in the evaluator.

Metrics and ablations. Net answer gain, harmful correction rate, unsupported answers, model/retrieval calls, wall time, duplicate tool effects and abstention. Ablate acceptance verification, rollback and budget selection. Include high-confidence wrong answers and corrections that add no evidence.

Proposed go/no-go gate. Keep only if it improves error reduction per extra call over fixed retry without increasing harmful corrections. A reasonable pilot gate is at least 15% fewer extra calls at matched final quality; confirm confidence intervals on conversation-clustered samples. A score-only improvement fails.

First task. Log paired before/after evidence and actual outcomes; stop using the change bonus as scientific proof of utility.

Disclosure needed. Specify permitted actions, observable acceptance tests, budget ownership, retry identities, rollback and tool-side-effect treatment. Document which rules are learned and which are fixed.

Guideline connection: original pp. 16-23, 31-41, 50-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P04, P05, P12, P13, X03, X04. Novelty and non-obviousness remain unproven.

R05. Version-safe online embedding and representation migration

Tier B | Technical fit: High | Readiness: Low-medium | Prior-art crowding: High | Effort: High

Code anchors: memory/embedding_pipeline.py:1; memory/embedding_rebuild.py:1; memory/memory_repository.py:1; memory/semantic_representation.py:1; memory/embedding_backend.py:1.

Implemented now. Embedding metadata fingerprints backend, model, dimension and representation/identity versions. A repository mismatch requires a rebuild rather than an online migration. An in-memory repository caches normalized vectors and uses exact cosine scanning; Chroma uses a different indexed backend. Query and note transforms share pipeline machinery.

Problem to investigate. An upgrade can require a pause or a costly full re-embedding. Comparing scores across incompatible vector spaces is unsafe. Merely checking a fingerprint, dual writing or rebuilding in batches is established practice.

Proposed mechanism. Build a bounded online migration that tracks per-note representation lineage and provides query coverage across old/new versions without mixing raw scores. Select migration order from query demand and dependency impact. Use version-local retrieval followed by a common verified reranker, then retire the old index only after coverage and consistency conditions are met.

Plain example. Move from an older embedding to a new model while questions continue; a recently queried note migrates first, but an unmigrated note remains retrievable through its old compatible query embedding.

Experiment and baselines. Compare stop-and-rebuild, ordinary dual-index migration, fixed-rate background migration and demand-aware migration. Include dimension and normalization changes, edited notes during migration and process restart. Start with one backend at a time before cross-backend comparisons.

Metrics and ablations. Re-embedding calls, bytes and peak disk/RSS, query latency, temporary quality loss, migration completion time, cross-version citation correctness and recovery work. Hold generator, corpus and reranker constant.

Proposed go/no-go gate. Proceed only if demand ordering reduces query disruption or work beyond the simple dual-index baseline while preserving version correctness. Do not claim the existing fingerprint check is an invention.

First task. Write a version-compatibility matrix and reproducible mismatch/rebuild benchmark.

Disclosure needed. Define compatibility, query fan-out, score comparability, write routing, cutover, garbage collection and recovery. Specify the actual hardware/storage constraints.

Guideline connection: original pp. 16-23, 35-41, 58-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P14, X02. Novelty and non-obviousness remain unproven.

R06. Recoverable commits across state, memory and derived indexes

Tier B | Technical fit: High | Readiness: Low-medium | Prior-art crowding: Very high | Effort: High

Code anchors: state/state_manager.py:1; workflow/prima_workflow.py:70; workflow/prima_workflow.py:485; workflow/prima_workflow.py:611; memory/maintenance/background_supervisor.py:1; events/event_bus.py:1.

Implemented now. State uses a process-local lock and optimistic version checks; JSON state persistence uses a temporary-file swap. State commit, memory commit and maintenance enqueue are separate phases. The maintenance queue and event deduplication sets are in memory, while terminal failure records can be durable. EventBus is an in-process sequential dispatcher.

Problem to investigate. A crash between phases can leave stores at different frontiers. Existing retry/deduplication is not proof of durable exactly-once execution. Conventional transaction logs, outbox patterns and idempotent consumers are strong routine baselines.

Proposed mechanism. Represent one accepted turn as a durable change set carrying expected state version and content-derived artifact versions. Recover its incomplete dependency work after restart, while preventing duplicate summaries or tool effects. Separate user-visible commit from derived-index readiness with an explicit consistency contract.

Plain example. Power fails after a note is stored but before graph maintenance is queued. Recovery discovers the committed change and rebuilds only its missing derived artifacts.

Experiment and baselines. Inject termination after every commit boundary and replay the same request/event ID. Compare current behavior, a standard transactional outbox and a proposed dependency-aware recovery scheme. Run single-process and multi-process configurations separately.

Metrics and ablations. Lost or duplicate changes, stale reads, recovery I/O/time, log growth, foreground latency and process-count scalability. Prove safety invariants for the supported failure model; do not claim all distributed failures are solved.

Proposed go/no-go gate. A mechanism beyond the standard outbox must save meaningful recovery work or preserve a stronger explicit consistency guarantee. Otherwise adopt the standard design as engineering work.

First task. Enumerate actual persistence boundaries and implement a failure-injection test harness before proposing a new protocol.

Disclosure needed. State crash model, log format, idempotency identity, commit points, ordering, multi-process assumptions and garbage-collection safety.

Guideline connection: original pp. 20-23, 35-41, 58; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P14, X02; extend search to transaction/outbox and materialized-view patents. Novelty and non-obviousness remain unproven.

R07. Measured resource scheduling with output-equivalence checks

Tier B | Technical fit: High | Readiness: Medium | Prior-art crowding: High | Effort: Medium

Code anchors: benchmarks/campaign/kaggle.py:135; benchmarks/campaign/provider_session.py:57; benchmarks/campaign/scheduler.py:1; benchmarks/campaign/comparison.py:1; PRIMA_Kaggle_Benchmark.ipynb:1.

Implemented now. Campaign tooling already estimates workload from observed rates, applies reserves, checks resources and supports sequential/parallel modes. A shared provider session bounds inference, caches identical requests and assigns workers to replicas. There are checks for output/evidence equivalence and actual overlap. Client construction time is explicitly not model-load time.

Problem to investigate. Predicted completion is not an observed full run. Parallel speedup can be inflated by response-cache hits, warm models, changed work or non-equivalent outputs. A fixed two-GPU notebook configuration is not a general scheduler.

Proposed mechanism. Use measured per-source demand and cache state to schedule isolated replay/QA work under RAM/VRAM and deadline constraints, with a runtime equivalence certificate for ordering-sensitive memory results. Preserve per-conversation order while allowing independent conversations to overlap; revise forecasts from observed contention.

Plain example. Two long conversations are placed on separate workers while their internal turns remain ordered; the scheduler admits a third workload only if live memory pressure and remaining-time estimates permit it.

Experiment and baselines. Compare serial, ordinary bounded thread pool, longest-work-first and adaptive resource scheduling on the same case IDs and model replicas. Report cold/warm/cache-disabled/cache-enabled regimes. Vary source skew and memory pressure; compare exact output/evidence only where deterministic inference makes that meaningful.

Metrics and ablations. Wall time, deadline error, actual overlap, RAM/VRAM peaks, cache hits, token counts, queue wait and output/evidence divergence. Record provider version and actual model loading separately.

Proposed go/no-go gate. Retain as research only if it improves scheduling under skew/contention beyond an ordinary queue while preserving the declared semantics. Existing resource guards alone are not a new contribution.

First task. Reproduce one small real paired campaign with complete traces; do not begin with an unsupported 12-hour promise.

Disclosure needed. Define hardware, resource signals, admission rules, forecast updates, cache regime and the precise equivalence guarantee.

Guideline connection: original pp. 16-23, 35-41, 58; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P16; extend search to inference scheduling and memory-constrained runtime scheduling. Novelty and non-obviousness remain unproven.

R08. Utility- and cost-aware memory admission

Tier B | Technical fit: Medium-high | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: memory/maintenance/memory_importance.py:75; config/memory_importance.yaml:1; workflow/prima_workflow.py:485; memory/maintenance/importance_score.py:1.

Implemented now. Admission uses weighted novelty, emotional intensity, user relevance, recurrence and reflection signals, with threshold 0.55 in the supplied configuration. Some recurrence work scans episodic memory. Accepted conversational notes and explicit imports follow different paths. Fixed lexical scores do not establish future retrieval utility.

Problem to investigate. Emotionally intense or repetitive text can occupy storage without helping future queries; rare low-affect facts can be critical. Learning CRUD decisions or weighted deterministic survival is already closely explored.

Proposed mechanism. Estimate the marginal downstream evidence value of admitting a note relative to storage, embedding and maintenance cost, with protections for unique dependency-bearing facts. Train on development replay outcomes and retain uncertainty/fallback for unseen query distributions.

Plain example. Keep a single change to a device's safe operating limit despite low emotional language; merge redundant greetings without losing source lineage.

Experiment and baselines. Compare admit-all, fixed current threshold, random-at-equal-size, recency/LRU, standard novelty sampling and compatible MemoryBank/Mem0/DMF or MemoryR1 configurations. Evaluate equal bytes as well as equal note counts; ablate emotional and recurrence features.

Metrics and ablations. QA/support recall at fixed bytes, embedding/maintenance calls, write latency, forgotten-critical-fact rate and distribution shift. Charge training and teacher-model cost separately.

Proposed go/no-go gate. A gain must survive equal-storage/equal-compute comparisons and outperform a simple novelty+recency rule. Otherwise it is parameter tuning in a crowded area.

First task. Log admission decisions with reason components and replay held-out future questions without leaking them into admission.

Disclosure needed. Describe feedback targets, data split, training/update procedure, unique-fact protection and resource accounting.

Guideline connection: original pp. 16-23, 35-43, 53-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P03, P10, P15, P17. Novelty and non-obviousness remain unproven.

R09. Dependency-preserving retention and forgetting

Tier B | Technical fit: Medium-high | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: memory/maintenance/consolidation_engine.py:1; memory/maintenance/retention_policy.py:1; memory/maintenance/forgetting_policy.py:1; memory/memory_lineage.py:1.

Implemented now. Retention multiplies importance by a fixed 0.96 per consolidation invocation. This is event-rate-dependent decay, not elapsed-time decay. Forgetting marks records rather than proving physical deletion of notes, vectors, graph edges and descendants. Repeated maintenance can therefore change retention even without new useful experience.

Problem to investigate. Routine maintenance frequency should not accidentally determine semantic lifetime. Time-based and interaction-count forgetting are both known; choosing either alone is a weak invention.

Proposed mechanism. Retain the minimal source/dependency set needed to support active facts and invalidate derived assertions when their support expires. Separate retention clocks from maintenance calls and bound storage reclamation while preserving a queryable audit trail.

Plain example. A superseded address can expire, but the evidence needed to answer "where did I live last year?" remains until its declared historical-retention policy expires.

Experiment and baselines. Replay identical conversations with different maintenance schedules. Compare current decay, wall-clock, interaction-count, LRU and dependency-aware forgetting at equal disk budgets. Test edits and obsolete summaries.

Metrics and ablations. Schedule sensitivity, critical-fact survival, temporal QA, actual reclaimed bytes, graph cleanup work and p95 query latency.

Proposed go/no-go gate. Require improvement beyond a clock fix and standard retention policies. Treat the clock fix and deletion correctness as baseline engineering.

First task. Build a schedule-invariance test and a physical-storage accounting report.

Disclosure needed. Specify clock semantics, support dependencies, archival versus deletion, version retention and descendant invalidation.

Guideline connection: original pp. 16-23, 35-43, 58-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P03, P14, P15. Novelty and non-obviousness remain unproven.

R10. Conservative identity resolution with session provenance

Tier B | Technical fit: Medium | Readiness: Medium | Prior-art crowding: High | Effort: Medium

Code anchors: memory/identity_normalization.py:1; memory/embedding_pipeline.py:1; memory/semantic_representation.py:1; evaluation/metrics/long_horizon_metrics.py:1; tests/evaluation/test_identity_consistency.py:1.

Implemented now. Identity normalization uses introductions/aliases and capitalized-name heuristics. The last named entity can replace third-person pronouns. Pipeline state is shared within a runtime and query embedding uses the same transformation machinery. Session ownership and ambiguous multi-person references need explicit testing.

Problem to investigate. A confident but wrong merge contaminates embeddings and later retrieval. Adding session keys or using a standard coreference model is necessary engineering but not by itself a strong invention.

Proposed mechanism. Store ambiguous identity candidates and source scopes without destructively rewriting raw text. Commit a normalization only under a reproducible evidence rule; propagate revisions to dependent representations while keeping raw retrieval available.

Plain example. In "Maya met Riya; she changed jobs", keep the subject ambiguous instead of silently assigning the job to the last name. A different session's person must not decide it.

Experiment and baselines. Compare normalization off, current heuristic, scoped heuristic, an established coreference baseline and deferred resolution. Hold embeddings constant and include aliases, multiple names, session switches and query-induced state changes.

Metrics and ablations. False merge/split rate, identity-specific recall, contaminated-answer rate, reindex work and latency. Evaluate by conversation/person-disjoint splits.

Proposed go/no-go gate. Require reduced damaging merges without an unacceptable recall or compute penalty beyond standard coreference/scoping.

First task. Add multi-person/session fixtures and test whether embedding a query mutates later identity normalization.

Disclosure needed. Describe ambiguity representation, scope ownership, decision thresholds and correction propagation.

Guideline connection: original pp. 16-23, 35-43, 59-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P02, P18; extend primary search to entity resolution and provenance revision. Novelty and non-obviousness remain unproven.

R11. Event-time temporal retrieval and validity tracking

Tier C | Technical fit: Medium-high | Readiness: Low-medium | Prior-art crowding: High | Effort: Medium

Code anchors: memory/retrieval/temporal_strategy.py:1; memory/retrieval/query_analysis.py:1; memory/event_memory/event_metadata.py:1; benchmarks/locomo/experiment.py:144; workflow/prima_workflow.py:485.

Implemented now. Temporal retrieval combines memory timestamps/age and lexical temporal constraints. Historical replay carries timestamp metadata, but the inspected note-creation path does not explicitly pass that historical timestamp as the note creation time. Event time, ingestion time and validity time are different concepts.

Problem to investigate. A replayed old fact can appear new; a later correction can invalidate a previous answer without erasing history. Time-aware query expansion and temporal benchmarks already exist.

Proposed mechanism. Index event and validity intervals separately from ingestion time, preserve supersession links and resolve queries against an explicit as-of time with bounded candidate filtering.

Plain example. "What was my role in March?" should use March's interval, not the time the conversation was imported.

Experiment and baselines. Compare current timestamps, correctly imported event timestamps, interval indexing and interval-plus-supersession. Use LongMemEval/LoCoMo temporal cases and synthetic out-of-order imports.

Metrics and ablations. Temporal QA/support recall, obsolete answers, temporal-filter candidates and latency.

Proposed go/no-go gate. First show the benefit beyond correcting timestamp plumbing; otherwise this is a correctness fix.

First task. Trace one historical timestamp from loader to final stored note and add out-of-order update fixtures.

Disclosure needed. Define event, ingestion and validity time, uncertain dates, interval boundaries and supersession behavior.

Guideline connection: original pp. 16-23, 35-41, 59-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P18, P14, X02. Novelty and non-obviousness remain unproven.

R12. Calibrated confidence for compute and abstention decisions

Tier C | Technical fit: Medium | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: uncertainty/confidence_aggregator.py:1; uncertainty/calibration.py:1; uncertainty/execution_gate.py:1; memory/retrieval/retrieval_confidence.py:1; evaluation/metrics/confidence_calibration.py:1.

Implemented now. Retrieval confidence is a fixed feature-weighted score. Aggregation uses fixed source weights, disagreement, heuristic intervals and decision normalization. Static calibration profiles are not evidence of empirical calibration. The simple evaluation aggregate accuracy-confidence gap is not a full expected calibration error measurement.

Problem to investigate. A confidence-like number can trigger needless reflection or overconfident answers. Temperature scaling and selective prediction are established methods.

Proposed mechanism. Calibrate decisions conditional on available evidence, task type and resource budget, then choose retrieval/correction/abstention under a measured risk constraint. Test a simple fitted calibrator before a complex controller.

Plain example. The same score can mean different error rates for a one-hop lookup and a multi-hop answer; a budgeted gate should account for that distinction.

Experiment and baselines. Compare fixed gate, development-fitted temperature/logistic/isotonic calibration and task-conditional control. Use a locked conversation-disjoint test set and a shifted dataset.

Metrics and ablations. Brier score, NLL where probabilities are valid, ECE with declared bins, reliability diagrams, risk-coverage curves and calls per correct answer. Do not call heuristic intervals confidence intervals.

Proposed go/no-go gate. Require better selective risk at matched cost beyond standard calibration. A better confidence chart alone is a weak patent proposition.

First task. Create a labeled outcome table and plot out-of-sample reliability of existing signals.

Disclosure needed. Define target event, fitting split, features, control mapping and domain-shift fallback.

Guideline connection: original pp. 16-23, 35-43, 53-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P05, P07, P21. Novelty and non-obviousness remain unproven.

R13. Postcondition-verified procedural memory and reuse

Tier C | Technical fit: Medium-high | Readiness: Low-medium | Prior-art crowding: Very high | Effort: High

Code anchors: workflow/prima_workflow.py:551; memory/memory_index.py:1; action/action_executor.py:1; reflection/rule_extractor.py:1.

Implemented now. A successful tool result can create a procedural note marked verified_success, containing natural-language procedure information and tool names. This records a success status; it is not a complete machine-checkable procedure with independent external postcondition verification. Explicitly imported procedures are another allowed path.

Problem to investigate. A previously successful action can be invalid under changed permissions, versions or world state. Generic experience extraction, reflection memory and success/failure memory are heavily disclosed.

Proposed mechanism. Store a typed procedure with preconditions, tool/schema versions, effect identities, independent postconditions and revocation dependencies. Reuse only after current-state compatibility checks; invalidate affected procedures when tools or permissions change.

Plain example. A file-conversion recipe is reusable only for the same supported format/tool version and only if the output is actually readable, not merely because the handler returned success.

Experiment and baselines. Compare no reuse, current note reuse, standard typed workflow caching and verified reuse with tool-version/permission changes and partial failures.

Metrics and ablations. Task success, invalid reuse, duplicate effects, validation cost and calls saved.

Proposed go/no-go gate. A new mechanism must outperform standard workflow caching and checks, and its claim must be distinguished from X03 and ExpeL.

First task. Define postconditions for two harmless local tools and collect failed-status/success-status mismatch examples.

Disclosure needed. Disclose schemas, compatibility rules, postcondition trust, effect identities, invalidation and recovery.

Guideline connection: original pp. 16-23, 31-41, 50-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P12, P13, X03. Novelty and non-obviousness remain unproven.

R14. Enforceable tool-effect and cancellation budgets

Tier C | Technical fit: High | Readiness: Low | Prior-art crowding: High | Effort: High

Code anchors: tools/tool_executor.py:1; action/execution_policy.py:1; workflow/orchestration_engine.py:229; security/permission_policy.py:1.

Implemented now. Allowlisting, schema checks, budget limits and sandbox flags exist. A sandbox label does not create operating-system isolation. The executor calls a synchronous handler before checking whether its result is awaitable; the asynchronous timeout therefore does not establish a bound on blocking synchronous work.

Problem to investigate. Cancellation or retry may leave an external effect running. Basic process isolation, idempotency keys and correct timeouts are established safeguards and must be baseline fixes.

Proposed mechanism. Explore a capability-bound effect ledger that coordinates cancellation, retries and postconditions under a resource/effect budget, using actual process/container boundaries for the stated threat model.

Plain example. A timeout during a file write must not permit an untracked second write when the workflow retries.

Experiment and baselines. Use local disposable tools with blocking, partial writes and cancellation. Compare current behavior, standard subprocess timeout/isolation and an effect-ledger controller. No live external side effects are needed.

Metrics and ablations. Untracked/duplicate effects, cancellation completion, containment escapes within the test model, overhead and recovery.

Proposed go/no-go gate. Security correctness is mandatory engineering; retain a patent research angle only if a specific non-routine coordination mechanism remains after standard safeguards.

First task. Document the actual execution boundary and fix the experimental harness before timing tools.

Disclosure needed. Define trusted components, OS boundary, capability enforcement, termination semantics and effect recovery.

Guideline connection: original pp. 16-23, 33-41, 58; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** X04; extend search to sandboxing, workflow idempotency and capability systems. Novelty and non-obviousness remain unproven.

R15. Affect uncertainty as a bounded compute-control signal

Tier C | Technical fit: Conditional | Readiness: Low | Prior-art crowding: High | Effort: Medium

Code anchors: affect/affect_engine.py:30; affect/retrieval_priors.py:1; affect/reflection_triggers.py:1; workflow/prima_workflow.py:146; planning/task_planner.py:1.

Implemented now. Affect computes PAD state, momentum, volatility, salience and reflection/retrieval hints with fixed rules. The current canonical evidence request does not pass these priors into retrieval scoring. Several priors appear as metadata rather than active ranking weights. The shared engine also keeps bounded history, which needs explicit session-scope review.

Problem to investigate. Empathy, mood-matching and preference-based personalization alone are weak under the highlighted exclusions. There is no current proof that affect improves retrieval or lowers compute.

Proposed mechanism. Only if supported by data, use affect uncertainty as one optional signal in a bounded controller for clarification or additional evidence, with session isolation and a task-only fallback. The technical claim would need a specific measurable compute/reliability mechanism.

Plain example. Ambiguous emotional phrasing could trigger one clarification instead of several speculative retrieval/reflection rounds; equally uncertain factual text should be a comparator.

Experiment and baselines. Compare task-only uncertainty, shuffled affect labels, current affect rules and calibrated affect-augmented control. Separate tone preferences from factual quality and resource metrics.

Metrics and ablations. Calls, latency, error/abstention, calibration and robustness across speakers/language styles. Do not infer clinical or psychological correctness from these labels.

Proposed go/no-go gate. Reject if gains disappear after task uncertainty is controlled, or amount only to perceived empathy. Scope isolation is an engineering prerequisite.

First task. Trace whether any proposed signal changes a real decision, then run a shuffled-signal ablation.

Disclosure needed. Describe labels, data traits, uncertainty, signal-to-control mapping, session state and robustness limits.

Guideline connection: original pp. 35-43, 49-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P03, P04, P22. Novelty and non-obviousness remain unproven.

R16. Adaptive event segmentation with raw-source fidelity

Tier C | Technical fit: Medium | Readiness: Medium | Prior-art crowding: High | Effort: Medium

Code anchors: memory/event_memory/event_segmenter.py:1; memory/event_memory/event_builder.py:25; memory/event_memory/event.py:1; memory/embedding_pipeline.py:1.

Implemented now. Event segmentation uses adjacency, overlap/bridge/shared-entity rules and a maximum of six turns. The event builder joins text, adds metadata and can repeat raw text in the embedding representation. Canonical single-turn creation is different from benchmark multi-turn segmentation.

Problem to investigate. Grouping can mix events or split their dependencies; repeated raw text can increase representation size. Event/session decomposition is already known.

Proposed mechanism. Select event boundaries using query-independent dependency continuity and a representation-size budget while preserving raw turn offsets. Keep cross-boundary links when a condition or identity depends on an earlier event.

Plain example. A correction in turn seven should remain connected to a fact in turn one even if the storage event has a six-turn limit.

Experiment and baselines. Compare raw turns, fixed-size chunks, current segmentation and adaptive boundaries at equal actual bytes/token budget.

Metrics and ablations. Boundary agreement, supporting-turn recall, QA, representation expansion and indexing/retrieval cost.

Proposed go/no-go gate. Require gains over fixed chunks with the same budget; a more readable summary is insufficient.

First task. Measure raw-to-event size ratio and document canonical versus benchmark entry points.

Disclosure needed. Specify boundary features, cross-event dependencies, source offsets and representation construction.

Guideline connection: original pp. 16-23, 35-41, 59-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P02, P18. Novelty and non-obviousness remain unproven.

R17. Query-conditioned graph paths with explicit support

Tier C | Technical fit: Medium-high | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: memory/graph/graph_reasoning_engine.py:1; memory/graph/graph_traversal.py:1; memory/retrieval/graph_strategy.py:1; reasoning/information_need_generator.py:1.

Implemented now. Graph reasoning uses degree centrality and Louvain communities or a fallback. Retrieval uses bounded graph traversal and weighted seeds; this is not learned relational inference. Existing neighbor caching and degree indexes already provide basic optimizations.

Problem to investigate. Highly connected nodes can be irrelevant, and a graph edge based on lexical overlap is not proof of a factual relation. Graph RAG and graph-walk retrieval are mature research directions.

Proposed mechanism. Explore typed, source-backed paths selected for the missing information need, pruning paths whose relation support is invalid or whose expansion cost exceeds budget.

Plain example. A path through two names sharing a hobby should not count as evidence that they work at the same company.

Experiment and baselines. Compare dense+sparse, existing graph strategy, simple typed-edge filtering and a compatible HippoRAG/LightRAG baseline. Separate retrieval recall from inferred-answer correctness.

Metrics and ablations. Supporting-path precision, expansion count, recall, QA and latency; ablate centrality and relation typing.

Proposed go/no-go gate. Simple relation validation must be beaten before claiming an inventive path-selection mechanism.

First task. Label a small path-support set and measure false relation assumptions.

Disclosure needed. Define edge semantics, source support, traversal/stop rules and update invalidation.

Guideline connection: original pp. 16-23, 31-41, 59-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P08, P09, P19, X01. Novelty and non-obviousness remain unproven.

R18. Support-checked semantic abstraction and lineage

Tier D | Technical fit: Conditional | Readiness: Low-medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: memory/evolution/semantic_abstraction_engine.py:10; memory/evolution/memory_evolution_engine.py:1; memory/evolution/lineage_tracker.py:1; memory/semantic_representation.py:1.

Implemented now. The current abstraction emits a sentence about a count of related memories and a keyword list. It is not a learned semantic abstraction system. Semantic representation adds structured fields and original text; it need not compress anything. Repeated eligible communities can produce repeated derived notes.

Problem to investigate. Calling this semantic learning would overstate the implementation. Cluster deduplication and source links are useful but ordinary; generative abstraction adds hallucination and model cost.

Proposed mechanism. Treat an abstraction as a materialized, source-supported view with explicit entailment scope and invalidation, rather than as a new fact. Investigate whether reusable support summaries reduce retrieval work under R01/R03.

Plain example. A summary of three project notes must identify what all sources support and must expire when a supporting source is corrected.

Experiment and baselines. Compare no abstraction, keyword abstraction, deduplicated extractive views and a source-checked summary at equal total storage and model cost.

Metrics and ablations. Unsupported derived claims, duplication, support coverage, bytes and retrieval latency/QA.

Proposed go/no-go gate. Retain primarily as part of R01/R03 unless a distinct mechanism survives A-MEM/Mem0/MemOS comparisons.

First task. Count identical source-set summaries across repeated evolution runs.

Disclosure needed. Describe view identity, support semantics, creation, invalidation, storage and model involvement.

Guideline connection: original pp. 20-23, 35-43, 53-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P02, P04, P10, P14, P20. Novelty and non-obviousness remain unproven.

R19. Emotion encoder and calibration pipeline

Tier D | Technical fit: Conditional | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium-high

Code anchors: affect/classifiers/goemotions_encoder.py:1; affect/classifier_factory.py:1; benchmarks/goemotions/training/losses.py:1; benchmarks/goemotions/training/calibration.py:1; benchmarks/goemotions/training/thresholds.py:1.

Implemented now. The encoder wraps existing models with multi-label scores, optional temperature and thresholds. Training contains established BCE/weighted/focal/asymmetric losses and development-set calibration/threshold selection. A hybrid factory name does not establish a new neural fusion architecture.

Problem to investigate. Changing a backbone, label threshold or familiar loss alone is usually a weak research distinction. Telemetry-only wrapper results do not establish improved emotion predictions.

Proposed mechanism. Investigate only a specific new deployment or preprocessing mechanism that yields reproducible technical benefit, such as bounded latency under label sparsity, and explain why standard quantization/calibration does not explain the result.

Plain example. A proposed sparse execution path would need to show fewer measured operations at matched macro-F1, not merely a better threshold on one validation split.

Experiment and baselines. Use official train/dev/test discipline; compare model-only, identical-model wrapper, standard calibration and the exact proposed change. Report neutral-label policy and class imbalance.

Metrics and ablations. Macro/micro F1, per-label precision/recall, calibration, latency, memory and training/inference cost.

Proposed go/no-go gate. Promote only with a new enabled mechanism beyond standard model training; dataset or threshold improvements alone remain lower priority.

First task. Freeze a reproducible model-only baseline and audit whether compared variants change predictions.

Disclosure needed. Record architecture, parameters, preprocessing, label traits, splits, training recipe, calibration and hardware integration if claimed.

Guideline connection: original pp. 16-23, 49-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P21, P22. Novelty and non-obviousness remain unproven.

R20. Planning and symbolic world constraints

Tier D | Technical fit: Conditional | Readiness: Medium | Prior-art crowding: Very high | Effort: Medium

Code anchors: planning/task_planner.py:24; planning/plan_evaluator.py:1; world/predictive_model.py:25; world/state_simulator.py:21; uncertainty/execution_gate.py:1.

Implemented now. Planning is deterministic and the world model applies symbolic action/constraint rules with fixed risk estimates. Simulation records predicted state transitions; it is not a learned physical simulator or calibrated prediction of external reality.

Problem to investigate. Imitating human reasoning or organizing conversational tasks is weak without a concrete technical process. Calling rule-based scoring a world model does not establish an inventive technical contribution.

Proposed mechanism. If the project later controls an actual technical process, specialize constraints to that process and verify predictions against observed states. For the present repo, use these modules to support bounded execution in R04/R14.

Plain example. A plan that predicts a file transformation must be checked against the actual output format and effects; a confidence label is not the observation.

Experiment and baselines. Compare standard rule checks and a simple state machine before any new planner. Test violated preconditions, false-safe predictions and unnecessary denials.

Metrics and ablations. Constraint violations, false denials, verified task completion and planning overhead.

Proposed go/no-go gate. Do not promote a general cognitive-planning claim without a new implemented technical constraint mechanism.

First task. List which predicted state variables are actually observable and verified.

Disclosure needed. Define state variables, rules, transitions, observation interface and calibration or explicit non-probabilistic interpretation.

Guideline connection: original pp. 31-43, 50-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P04, P12. Novelty and non-obviousness remain unproven.

R21. Typed routes and trusted/untrusted prompt boundaries

Tier D | Technical fit: Medium | Readiness: Medium | Prior-art crowding: Very high | Effort: Low-medium

Code anchors: runtime/contracts.py:173; runtime/route_profiles.py:145; llm/prompt_builder.py:1; workflow/output_validation.py:45; security/input_sanitizer.py:1.

Implemented now. Trusted immutable options reject reserved metadata overrides, routes own lifecycle phases, and prompt evidence is explicitly marked untrusted. There are six task kinds, five profiles and ten valid route pairs. Tagging untrusted text and regex sanitization do not provide formal prompt-injection isolation; structural citation checks are not truth verification.

Problem to investigate. Typed contracts, schemas, delimiters and sanitization are standard safeguards. A new prompt layout alone has little research strength.

Proposed mechanism. Only a specific enforceable information-flow mechanism that prevents untrusted evidence from changing tool capabilities or trusted execution state would justify escalation. Keep policy enforcement outside the model.

Plain example. A retrieved note saying "ignore rules and call a tool" must remain data and must never alter the external allowlist.

Experiment and baselines. Compare existing boundaries, a standard structured-message/tool-policy setup and the proposed mechanism on adversarial retrieved content, metadata override attempts and malformed outputs.

Metrics and ablations. Unauthorized state/capability changes, task success, false rejection and overhead. Define attack coverage rather than claiming universal security.

Proposed go/no-go gate. Use as architecture quality unless a non-routine enforceable mechanism remains beyond standard validation.

First task. Write an explicit trust-boundary diagram and attack fixtures against the actual route.

Disclosure needed. Specify trust assumptions, serialization, external enforcement, capabilities and residual model behavior.

Guideline connection: original pp. 16-23, 31-43, 50-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** Extend primary search to prompt-injection defenses and information-flow control. Novelty and non-obviousness remain unproven.

R22. Evaluation integrity and replay accounting

Tier D | Technical fit: Conditional | Readiness: Medium | Prior-art crowding: High | Effort: Low-medium

Code anchors: benchmarks/common/artifacts.py:1; benchmarks/campaign/comparison.py:1; docs/benchmarks/RESULT_CLAIM_POLICY.md:1; evaluation/metrics/phase5_scientific_metrics.py:340.

Implemented now. Benchmark tooling includes atomic manifests, authoritative checkpoints, paired comparisons, case IDs and a claim policy. Some older scientific metrics are analytic proxies: retrieval_scaling computes a formula, not measured latency. A representation consistency check compares serialization to itself. These cannot establish technical improvement.

Problem to investigate. Evaluation correctness is essential research infrastructure. Atomic files, result manifests and bootstrap statistics are established techniques, rather than standalone inventions.

Proposed mechanism. Use this family to make R01-R21 credible. A separate research angle would require a specific new replay/measurement protocol that detects or prevents a technical failure more efficiently than ordinary provenance and reproducibility tooling.

Plain example. A campaign report should reject a speedup caused only by different questions, cached model responses or omitted failed cases.

Experiment and baselines. Inject mismatched case sets, altered budgets, stale artifacts and interrupted writes, then test detection and reconstruction.

Metrics and ablations. Detected mismatches, artifact loss, reconstruction work and instrumentation overhead.

Proposed go/no-go gate. Treat primarily as enabling work; do not patent a scorecard or presentation of results.

First task. Replace proxy performance claims with actual traces and publish the evaluation denominator.

Disclosure needed. Specify execution identity, artifact authority, comparison prerequisites and failure accounting.

Guideline connection: original pp. 16-23, 35-43; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** P18; extend search only if a distinct measurement mechanism is found. Novelty and non-obviousness remain unproven.

R23. Provider plumbing, configuration, events and monitoring

Tier D | Technical fit: Low standalone | Readiness: High for reuse | Prior-art crowding: Very high | Effort: Low

Code anchors: llm/llm_client.py:1; llm/rate_limiter.py:1; events/event_bus.py:1; monitoring/telemetry.py:1; config/runtime_config.py:1; main.py:1.

Implemented now. These modules supply provider requests/retries, rate limiting, event dispatch, configuration, health and telemetry, plus entry-point wiring. They support technical experiments but no distinctive mechanism was established by this audit.

Problem to investigate. A generic wrapper, dataclass, environment setting, event callback or dashboard is weak as a standalone patent research direction.

Proposed mechanism. Reuse these components as measurement and execution infrastructure. Revisit only if an experiment exposes a narrow new resource-control or consistency mechanism that maps to R06/R07/R14.

Plain example. A timer can record a retrieval improvement; adding the timer is not the retrieval invention.

Experiment and baselines. Check instrumentation overhead and whether relevant failures are observable; use ordinary library behavior as baseline.

Metrics and ablations. Trace completeness, overhead and reproducibility, with secrets redacted.

Proposed go/no-go gate. Do not spend the first research cycle inventing replacements for standard plumbing.

First task. Reuse existing telemetry and establish one coherent run manifest.

Disclosure needed. If later included in an invention, explain its concrete functional connection rather than listing component names.

Guideline connection: original pp. 16-23, 35-43; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** Standard-library/provider documentation and established systems techniques; no novelty claim. Novelty and non-obviousness remain unproven.

R24. Third-party assets, fixtures, exports and project documents

Tier D | Technical fit: No team-original mechanism established | Readiness: Support only | Prior-art crowding: Not assessed as team IP | Effort: Low

Code anchors: .gitmodules:1; LICENSE:1; CONTRIBUTORS.md:1; requirements.txt:1; tests/conftest.py:1.

Implemented now. External LoCoMo, LoCoMo-v2 and HotpotQA trees, pretrained models and their datasets are external work. Tests, fixtures, export files, setup/CI scripts, notebooks and documents either verify or package the project. Owned benchmark adapters are classified separately in R07/R19/R22.

Problem to investigate. A folder being inside the repository does not establish that the team invented its content. An inventory must distinguish technical contributions, support code and third-party material.

Proposed mechanism. Use external assets lawfully as declared baselines and evaluation resources. Record original human contributions to any later mechanism, rather than claiming dataset/model ownership.

Plain example. Using a GoEmotions model and a LoCoMo dataset can support an experiment; their established architecture/data are not PRIMA's invention.

Experiment and baselines. Audit provenance, licenses, dataset splits and the exact baseline implementation used. Check fixture leakage into training/test evaluation.

Metrics and ablations. Complete provenance and reproducibility records; no patent score is inferred from file count.

Proposed go/no-go gate. Keep these assets outside a team-original invention claim unless independently documented new human technical work is identified.

First task. Build the disclosure/inventorship chronology described in this report.

Disclosure needed. Record authors, licenses, public dates, contribution boundaries and essential dataset traits without inventing ownership.

Guideline connection: original pp. 16-23, 49-50, 59-61; technical effect, separate statutory tests and enablement remain necessary. **Closest screen references:** See baseline sources and repository licenses. Novelty and non-obviousness remain unproven.

15. Primary research screen

Dates below are first arXiv submissions where available, or the stated publication month. They are not an assessment of every earlier public version. Abstract-level scope was used for this screen; full-method comparison remains a team task. Descriptions identify crowding, not established anticipation of a final claim.

P01. MemGPT

First listed public version: 2023-10-12. Relevant families: R01, R02. Hierarchical external memory and control flow are established. Generic memory tiers or context management are weak distinctions.

P02. A-MEM

First listed public version: 2025-02-17. Relevant families: R01, R08, R10, R16-R18. Structured notes, dynamic links and evolution of historical memories closely overlap the project's broad memory story.

P03. MemoryBank

First listed public version: 2023-05-17. Relevant families: R08, R09, R15. Time/significance-based forgetting and reinforcement narrow claims around personalized memory and retention.

P04. Generative Agents

First listed public version: 2023-04-07. Relevant families: R04, R15, R18, R20. Memory, retrieval, reflection and planning already appear together. Combining these labels is not a new architecture.

P05. Corrective Retrieval Augmented Generation

First listed public version: 2024-01-29. Relevant families: R02, R04, R12. Retrieval-quality evaluation and corrective actions crowd confidence-triggered retry designs.

P06. Self-RAG

First listed public version: 2023-10-17. Relevant families: R02, R04. Adaptive retrieval and reflection-based assessment crowd generic retrieve/critique loops.

P07. Adaptive-RAG

First listed public version: 2024-03-21. Relevant families: R02, R12. Question-complexity-dependent retrieval strategies crowd broad query-adaptive routing.

P08. HippoRAG

First listed public version: 2024-05-23. Relevant families: R02, R17. Graph-based retrieval with Personalized PageRank makes generic associative graph memory a crowded direction.

P09. LightRAG

First listed public version: 2024-10-08. Relevant families: R01, R17. Graph retrieval and incremental updates are already explored. Incrementality alone cannot be the claimed difference.

P10. Mem0

First listed public version: 2025-04-28. Relevant families: R02, R08, R18. Dynamic memory extraction, consolidation and retrieval, including a graph variant, crowd broad memory-layer claims.

P11. LongLLMLingua

First listed public version: 2023-10-10. Relevant families: R03. Question-aware context compression is established; compare matched token/quality budgets.

P12. Reflexion

First listed public version: 2023-03-20. Relevant families: R04, R13, R20. Linguistic feedback and episodic reflection for later attempts crowd generic self-improvement loops.

P13. ExpeL

First listed public version: 2023-08-20. Relevant families: R04, R13. Knowledge extraction from experience and reuse make generic learned procedural notes a weak distinction.

P14. MemOS

First listed public version: 2025-07-04. Relevant families: R01, R05, R06, R09, R11, R18. Unified memory lifecycle, scheduling, provenance and versioned MemCubes crowd broad memory-OS/versioning claims.

P15. DMF

First listed public version: 2026-06-02. Relevant families: R01, R08, R09. A deterministic CPU-first memory pipeline combines NLP/vector signals, provenance scoring and interaction-count decay. Deterministic heuristics or zero generative write calls alone are weak distinctions.

P16. Sleep-time Compute

First listed public version: 2025-04-17. Relevant families: R01, R07. Offline context processing and amortization crowd generic background-computation claims.

P17. Memory-R1

First listed public version: 2025-08-27. Relevant families: R08. Outcome-driven reinforcement learning for memory management crowds broad learned admission/CRUD policies.

P18. LongMemEval

First listed public version: 2024-10-14. Relevant families: R10, R11, R16, R22. Long-term memory evaluation includes temporal reasoning, updates and abstention; session decomposition and time-aware query expansion are existing comparison points.

P19. From RAG to Memory / HippoRAG 2

First listed public version: 2025-02-20. Relevant families: R02, R17. A relevant non-parametric memory/retrieval comparator. Inspect the full method/version before asserting any feature-level difference.

P20. The Attribution-Compression Frontier in RAG

First listed public version: 2026-09-13. Relevant families: R03, R18. Directly studies compression versus original-source attribution, beyond citation presence. Its abstract also acknowledges evaluator dependence and lack of independent human calibration; PRIMA must not treat automated support scores as ground truth.

P21. On Calibration of Modern Neural Networks

First listed public version: 2017-06-14. Relevant families: R12, R19. Temperature scaling is established. A fitted temperature is not an original architecture.

P22. GoEmotions

First listed public version: 2020-07. Relevant families: R15, R19, R24. The established fine-grained emotion dataset and label taxonomy are external resources, not a team-original invention.

16. Patent-publication screen and preliminary feature comparison

These are published records used as technical disclosures. Selected claim passages were inspected; no legal-status, validity, enforceability, Indian-family or freedom-to-operate conclusion is made. Publication dates below must not be confused with priority dates. The final mechanism needs a full feature chart and family search.

X01. US20210109952A1: Incremental clustering for enterprise knowledge graph

Publication: 2021-04-15. **Inspected feature overlap:** Claim 1 links template/instance extraction, entity clustering, candidate queries and graph updates. Claim 2 addresses batches sized for memory footprint; dependent claims address uncertain entity linking.

Research consequence: R01/R17 must distinguish a concrete dependency/query-consistency mechanism from ordinary incremental clustering and memory-bounded batches. No absence of such features is established by this screen.

X02. US20230342636A1: Constructing knowledge graph data

Publication: 2023-10-26. **Inspected feature overlap:** Claim 1 concerns graph differences, time-based versions, node/edge mapping and storage of versioned incremental data.

Research consequence: R01/R05/R06/R11 cannot rely simply on versioned graph deltas. Inspect the entire family, earlier publications and relevant claims before forming a novelty position.

X03. US20260111708A1: Management of long-term memory recall through a self-reflection protocol

Publication: 2026-04-23. **Inspected feature overlap:** Claim 1 combines a task, reflection over communications, stored samples and later recall for a later task. Dependent claims include categories, preferences, success/failure labels and user confirmation/edit permissions.

Research consequence: R04/R13 need much more than storing reflections or a successful procedure. Compare exact postcondition/version/effect coordination features, not new terminology.

X04. US20260134212A1: LLM-reflection based adaptive prompt correction in multistage workflows

Publication: 2026-05-14. **Inspected feature overlap:** Claim 1 includes detected errors, iterative collaborative scores, structured prompt parts and replacing a selected part to produce a revised prompt. Dependent claims cover threshold finalization and multistage operation.

Research consequence: R04/R14 must distinguish the exact observable acceptance/effect-budget mechanism from iterative score-driven prompt correction.

17. Your 56 highlights: requirement-to-research crosswalk

Guide p. means the annotated 27-page study guide; source pp. means the original 67-page guidelines. Short topics below identify your highlights; they are not verbatim quotations of the original law. Adjacent quadpoint crops can contain fragments, so the original provision/page controls. All 56 highlight IDs are accounted for.

Highlight / guide p.	Topic	Original source pp.	Research action / family
H01 / 3	Useful app is not enough	9, 16-23	All: Require eligible implemented contribution and every ordinary patent test.

Highlight / guide p.	Topic	Original source pp.	Research action / family
H02 / 3	Four separate 3(k) exclusions	9-10, 23-39	All: Check each applicable exclusion separately.
H03 / 3	Abstract math versus implementation	23-34	R01-R14: Describe executable components and technical operation, not a formula alone.
H04 / 3	How and best method	20-23	All: Write reproducible structures, steps, failures and best implementation.
H05 / 3	Architecture versus data contribution	53-61	R05, R08, R19: Disclose what actually supplies the effect.
H06 / 3	Human inventorship	50	All: Record human conception and AI assistance.
H07 / 5	AI/ML/DL meaning	6, 49-50	R19, R20: Do not call fixed rules a learned neural architecture.
H08 / 5	Architecture/layers/parameters	53-61	R19: Describe essential model/training details if claimed.
H09 / 6	Attention/embeddings/time	53-61	R05, R11, R19: Explain actual representation and temporal mechanism.
H10 / 6	Preprocessing and segmentation	55-56, 59-61	R10, R16, R19: Connect raw input transformations to final effect.
H11 / 7	Accuracy/recall/errors/robustness	53-61	All experimental tracks: Use appropriate held-out metrics and failure accounting.
H12 / 9	Novelty steps 1-6	16-17	All: Compare essential features and earlier public disclosures.
H13 / 10	Novelty documentation step 7	16-17	All: Save exact prior-art passages, dates and reasons.
H14 / 10	Non-obviousness also required	17-19	All: Technical advance/economic value does not replace inventive step.
H15 / 10	Five-step inventive assessment	18-19	All: Identify skilled person, knowledge, concept, differences and obviousness.
H16 / 10	Industrial applicability	20	All: Specify a workable technical deployment.
H17 / 14	Internal computation	39-41	R01, R02, R07: Measure actual operations and processor time.
H18 / 14	Storage access	39-41	R01, R05: Measure I/O and access latency beyond ordinary caching.
H19 / 14	Memory management	39-41	R01, R08, R09: Measure storage/maintenance and preserved quality.
H20 / 14	Database operation	39-41	R01, R05, R06: Explain indexing, consistency and recovery mechanisms.
H21 / 14	Compression/expansion	39-41	R03: Use actual tokens/bytes and original-source fidelity.
H22 / 14	Authentication/privacy/security	39-41	R14, R21: Require enforceable mechanisms; labels are insufficient.

Highlight / guide p.	Topic	Original source pp.	Research action / family
H23 / 14	Resource allocation	39-41	R01, R02, R07: Measure resource-bound scheduling against standard queues.
H24 / 14	Error detection/correction	39-41	R04, R06: Measure actual errors/recovery, not score changes.
H25 / 14	Machine control	39-41	R14, R20: No actual physical control established in this repo.
H26 / 14	Signal handling	39-41	R19 conditional: No new signal-processing mechanism established.
H27 / 14	Real-time monitoring/control	39-41	R07, R14 conditional: Measure deadlines and real controls if claimed.
H28 / 14	Embedded control	39-41	No current direct implementation: Do not add a hardware label without actual integration.
H29 / 15	Autonomous vehicle/drone	39-41	No current direct implementation: Outside present implementation; not a candidate by analogy alone.
H30 / 15	Medical image analysis	39-41	No current direct implementation: Emotion classification is not evidence of medical imaging innovation.
H31 / 15	Distributed/cloud systems	39-41	R06, R07: Distinguish process-local behavior from distributed guarantees.
H32 / 15	Technical simulation	39-41	R20 conditional: Symbolic predictions need a technical process and validation.
H33 / 15	Efficient ML/architecture	39-41	R05, R19: Require a specific new mechanism and measured compute/quality.
H34 / 15	Technical signal processing	39-41	No current direct mechanism: Do not infer a signal invention from existing encoders.
H35 / 15	Human-reasoning imitation	41-43	R04, R20: Ground control in technical operation and evidence.
H36 / 15	Mental/game rules	9, 41-43	R20, R21: A thinking/workflow rule is not enough.
H37 / 15	Ordinary automation	41-43	R21, R23: Show an additional technical mechanism/effect.
H38 / 16	Standard lookup	41-43	R02, R11, R17: Beat ordinary retrieval with fair cost/quality accounting.
H39 / 16	Preference personalization	41-43	R08, R15: Empathy/preferences alone are a weak technical basis.
H40 / 16	Algorithm flowchart	46	All: Enabled technical implementation and effect matter.
H41 / 16	Program-per-se flowchart	47-48	All: Interpret whole claim and assess essential technical character.
H42 / 17	AI-assisted versus AI-generated	50	All: Keep the human contribution record.
H43 / 17	Named parts without integration	50-53	All: Teach concrete connections and operation, not box names.

Highlight / guide p.	Topic	Original source pp.	Research action / family
H44 / 18	Transformation and validating evidence	53-54	R03, R16, R19: Reproduce the input-output transformation and technical result.
H45 / 18	Data/model/training/reliability	54-55	R08, R12, R19: Specify essential data and training traits and reliable outcomes.
H46 / 19	Preprocessing connection	55-56	R05, R10, R16, R19: Disclose every material step and its role.
H47 / 19	RL state/action/reward	56-57	R02, R04, R08 if learned: No RL claim without its actual observation/action/reward design.
H48 / 20	Internal hardware/system performance	58	R01-R07: Explain integration and measure actual internal metrics.
H49 / 20	Essential dataset traits	59	All learning tracks: Record domain, diversity, labels and defining traits.
H50 / 21	Learning implementation versus data	60-61	R05, R08, R19: Match disclosure to the claimed inventive contribution.
H51 / 24	Prompt/workflow reading aid	31-43, 49-61	R04, R21: Guide synthesis, not a dedicated prompt patent rule.
H52 / 24	RAG/memory reading aid	35-43, 58-61	R01-R03, R05-R11: Ordinary lookup differs from an implemented technical improvement.
H53 / 24	Fine-tuning/data reading aid	53-61	R08, R19: Generic tuning is weak; disclose essential mechanism/data traits.
H54 / 24	Agents/control reading aid	56-58	R04, R13, R14, R20: Explain actual observation, action and effect interface.
H55 / 24	Architecture connection reading aid	20-23, 50-53	All: A memory/reasoning/learning block diagram is not enablement.
H56 / 24	Labels do not change tests	16-23, 34-43, 49-61	All: Apply all statutory and disclosure checks to the final mechanism.

18. Decision records and ready-to-use team templates

For each surviving candidate, complete the following fields in a shared record. Keep failed candidates and why they failed. This prevents a later patent narrative from erasing important negative evidence.

Candidate and owner. Research ID, human owner, dated version, technical collaborators and reviewer.

Problem and mechanism. One observable technical bottleneck/failure; the smallest proposed operations and data structures; what currently exists and what is new.

Closest alternatives. Current code, standard baseline, closest full papers/patent families and exact feature chart.

Predicted effect. Primary resource/correctness metric, practical minimum effect, quality tolerance and scope assumptions.

Experimental identity. Run manifests, source/model/dataset/config hashes, hardware, splits, seed, budgets and cache regime.

Outcome and uncertainty. All-case results, paired clustered intervals, failures, ablations, cost and limits.

Enablement. Best implementation, pseudocode, component connections, failure/recovery behavior, essential data and training traits.

Disclosure and inventorship. Human conception/contribution log; earlier publications/presentations/repository visibility; college/owner review.

Decision. Continue, revise, engineering-only, paper/capstone-only or patent-assessment packet; reason and next review trigger.

19. Completion and limits of this audit

All 385 owned Python files are in the ledger, all source roots are in the module matrix, all 24 research families have an experiment and disposition, and all 56 highlights are mapped to original-guideline pages. Code hashes were checked against the structural inventory before authoring. The saved guide and original PDF were not edited.

This is a deep research foundation, not a proven invention selection. Exhaustive worldwide prior art, patent-family/legal-status/FTO review, public-disclosure chronology, ownership determination, full current integration tests and new real-model/GPU benchmark campaigns remain uncompleted. The report states these limits where they affect decisions. The team should choose a narrow track using evidence and be willing to stop a patent angle when ordinary engineering or earlier work explains the result.

20. Source authority and official references

Original supplied document: C:/Users/Yuv Nahar/Downloads/Indian_patent_guidelines_CRI-AIML.pdf (67 pages). User-annotated study guide: C:/PRIMA_next/output/pdf/Indian_CRI_AI_ML_Plain_Language_Guide.pdf (27 pages, 56 highlights). These source files are preserved.

Official publication context: IP India guidelines index and 2025 CRI guidelines. This report applies the supplied document; it does not conflate it with later guidelines about using AI within patent examination.